

Text Summarization Tool for Looma Education

Rahul Gupta

July 2024

Introduction

In this report, I will explain the text summarization tool I developed for Looma Education. The tool extracts text from PDF files, generates summaries using an AI model, and shows the summaries on a web page. I will talk about the packages I used, the functions I wrote, and provide some useful links.

Installation and Setup

Here are the steps to set up and run the tool:

- Install Python from Python's official website.
- Install MongoDB from MongoDB's official website.
- Install Tesseract OCR from Tesseract's GitHub page.
- Install the required Python packages using pip:

```
pip install flask pymupdf pdf2image pytesseract pymongo
```

- Download the project files from the GitHub repository: LoomaAI.
- Start the MongoDB server.
- Set up Ollama for the LLaMA3 model as per the instructions from Ollama's documentation.
- Run the Flask server:

```
python app.py
```

- Open a web browser and go to `http://localhost:5001` to use the tool.

How It Works

The text summarization tool uses a client-server setup. There are two main parts: the client and the server.

- **Server:**
 - **Flask Web Server:** I used Flask to create the web server. Flask helps manage HTTP requests and responses. I set up routes in Flask to handle things like generating summaries and showing the chatbox.
 - **MongoDB:** I used MongoDB to store information about textbooks and chapters. The server asks MongoDB for data like chapter details and PDF file paths.
 - **Text Extraction:** The server uses PyMuPDF (fitz) to get text directly from PDF files. If the PDF has scanned images, the server uses Tesseract OCR to convert the images to text. This way, I can get text from both text-based and image-based PDFs.
 - **Summarization Model:** The server talks to a pre-trained AI model (like Meta LLaMA) to create text summaries. The server sends the extracted text to the model, and the model sends back a summary.
- **Client:**
 - **Web Interface:** The web interface is an HTML page with a chatbox where users can type in chapter IDs and get summaries. JavaScript handles user input, sends requests to the server, and shows the responses.
 - **AJAX Requests:** JavaScript sends requests to the Flask server's `/generate_summary` endpoint. It sends the chapter ID and gets the summary back, which is then shown in the chatbox.

Packages and Libraries

Here are the main Python packages and libraries I used:

- **Flask:** To create the server and manage HTTP requests and responses.
- **PyMuPDF (fitz):** To get text from PDF files.
- **pdf2image:** To convert PDF pages to images.
- **pytesseract:** To perform OCR on images.
- **pymongo:** To interact with MongoDB.

Functions

Here are the main functions I wrote for the tool:

- **extract_text_from_page**: This function tries to get text directly from a PDF page. If it finds text, it returns it. If not, it returns **None** to show that OCR is needed.
- **perform_ocr**: This function performs OCR on an image to get text using Tesseract. I use this when direct text extraction from the PDF page doesn't work.
- **extract_text_from_pdf**: This function downloads the PDF and gets text from the specified pages. It first tries to get text directly using PyMuPDF. If that fails, it converts the pages to images and uses OCR to get the text from the images.
- **llama3**: This function sends the extracted text to the LLaMA AI model to get a summary. It sends a POST request to the model's API with the text and gets back the summary.

Resources and Links

Here are some useful resources and links:

- **GitHub Repository**: <https://github.com/Sciotaku/LoomaAI>
- **Flask Documentation**: <https://flask.palletsprojects.com/en/2.0.x/>
- **PyMuPDF Documentation**: <https://pymupdf.readthedocs.io/en/latest/>
- **pdf2image Documentation**: <https://pdf2image.readthedocs.io/en/latest/>
- **Tesseract OCR**: <https://github.com/tesseract-ocr/tesseract>
- **PyMongo Documentation**: <https://pymongo.readthedocs.io/en/stable/>

Acknowledgments

I would like to acknowledge professor Peter Chin for his guidance in using the Llama3 model. I would also like to acknowledge Atulya and Praneel for their code 'generate.py' and 'split.py'.